

中山大学计算机学院本科生实验报告

(2024 学年春季学期)

课程名称：并行算法

批改人：

实验	使用 C++ 实现并行排序算法	专业（方向）	信息与计算科学
学号	22336049	姓名	陳日康
Email	chenih5@mail2.sysu.edu.cn	完成日期	2025 年 4 月 15 日

1. 实验内容

使用 C++ 实现两种并行排序算法，以掌握并行编程的基本方法，并通过实验分析其性能提升情况。要求从奇偶归并排序、双调归并排序、Valiant 并行归并排序中选择一种作为并行归并排序算法，同时从奇偶移项排序、Shear 排序、并行快速排序中选择一种作为其他类型的并行排序算法。

实现过程中可使用 C++11/17 线程库、OpenMP、MPI 或 CUDA 等并行工具，排序算法需在逻辑上正确，并能处理至少 10^6 数量级的数据。程序必须采用多线程或多进程实现，明确说明任务划分、数据分块、线程管理等并行化策略。实验需在不少于 5 组不同规模的随机数据（如 10^5 、 10^6 、 10^7 等）上进行，并对负载均衡情况和两种算法的性能差异进行对比分析。

2. 算法原理

(i). 双调归并排序

双调归并排序（Bitonic Sort）是一种基于分治法的排序算法，它通过将数组分成两部分，一部分按升序排序，另一部分按降序排序，最终将整个数组变为双调序列。通过这种方式，数组逐渐变得接近有序，最终通过合并操作得到一个完全排序的数组，而这个合并过程可以并行化。

在并行化方面，双调归并排序通过在递归的每个步骤中为左右两部分创建线程来加速排序。当递归深度较小且任务较大时，会创建多个线程来并行处理；当递归深度较深时，则转向串行处理。任务通过将数组分割为较小的部分并分配给不同线程来划分，每次递归中数组会被均分为两部分，左半部分按升序排序，右半部分按降序排序，从而加速排序过程。通过这种方式，算法能够充分利用多核处理器的计算能力，提高排序效率。

(ii). 并行快速排序（Parallel QuickSort）

并行快速排序（Parallel QuickSort）通过多线程加速排序过程，选择一个基准元素，将数组分为两部分，然后递归地对每个部分进行排序。在并行快速排序中，当数组被分割成多个部分时，可以通过在不同的线程中并行处理这些子数组来提高排序效率。

并行快速排序的并行策略主要体现在递归的并行化上。每当数组分割成两个子数组时，算法会为这两个子数组创建不同的线程进行处理。这样，多个线程就能同时处理不同部分的排序工作，缩

短整体执行时间。为了避免过多线程带来的开销，通常会根据递归的深度来控制线程的创建。当递归深度较小且任务较大时，会启用并行处理；而当递归深度较深时会转向串行处理。任务的划分主要依赖于数组的分割。在每次递归时，数组会被分为两个部分，分别在两个线程中排序。线程的数量和任务的大小通常会动态调整，以确保负载均衡并最大化并行效率。

3. 关键代码解析

(1). C++ 实现双调归并排序（1_bitonic_sort.cpp）

(i). bitonicMerge 函数

```
void bitonicMerge(std::vector<int>& arr, int low, int cnt, bool dir) {
    if (cnt > 1) {
        int k = cnt/2; // 拆分成两个部分进行比较
        for (int i=low; i<low+k; ++i) {
            // 根据排序方向交换数据
            if (dir == (arr[i] > arr[i+k])) {
                std::swap(arr[i], arr[i+k]);
            }
        }
        bitonicMerge(arr, low, k, dir); // 递归合并前半段
        bitonicMerge(arr, low + k, k, dir); // 递归合并后半段
    }
}
```

对数组中一段构成 Bitonic 序列的元素进行合并操作，使其完全有序。Bitonic 序列是先升后降（或反之）的序列。合并过程中对比相距 k 的两个元素，根据 dir 决定是否交换。再递归分别合并前后两个子序列。

(ii). bitonicSort 函数

```
void bitonicSort(std::vector<int>& arr, int low, int cnt, bool dir, int depth = 0) {
    printThreadOnce();

    if (cnt > 1) {
        int k = cnt/2; // 将数组分成两段

        if (depth < MAX_THREAD_DEPTH) {
            // 在可用线程范围内，新开线程排序左半段
            std::thread t1(bitonicSort, std::ref(arr), low, k, true, depth+1);
            bitonicSort(arr, low+k, k, false, depth+1); // 当前线程处理右半段
            t1.join(); // 等待子线程完成
        }
        else {
            // 递归调用
            bitonicSort(arr, low, k, true, depth+1);
            bitonicSort(arr, low+k, k, false, depth+1);
        }

        // 合并排序结果
        bitonicMerge(arr, low, cnt, dir);
    }
}
```

`bitonicSort` 负责递归地将数组划分为更小的子序列进行排序，并在合适的递归层级启动线程以实现并行处理。当子序列长度 cnt 为 1 时，递归终止，直接返回。函数将当前序列划分为左半段和右半段，分别进行升序和降序排序。若当前递归深度 $depth$ 小于预设的最大线程深度 `MAX_THREAD_DEPTH`，则创建一个新线程 `t1` 来处理左半段升序排序，而当前线程负责右半段降序排

序，排序完成后通过 `join()` 等待子线程执行结束。若递归深度已达到最大限制，则均由当前线程串行递归处理。递归排序完成后，调用 `bitonicMerge` 对当前子序列进行合并，以确保局部有序性。通过控制 `MAX_THREAD_DEPTH`，可以限制并行的层级，防止线程过多造成资源竞争和调度开销。

(2). C++ 实现并行快速排序（2_parallel_quicksort.cpp）

(i). partition 函数

```
int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high], i = low-1;
    for (int j=low; j<high; ++j) {
        if (arr[j] < pivot)
            std::swap(arr[++i], arr[j]);
    }
    std::swap(arr[i+1], arr[high]);
    return i+1;
}
```

将数组中的一段划分成两部分，左边是所有小于基准值的元素，右边是大于等于基准值的元素。这个函数选取的基准值是当前区间的最后一个元素 `arr[high]`，然后从 `low` 到 `high-1` 遍历数组，用变量 `i` 记录当前已知小于基准值的区域的最后一个下标。每当遇到一个比基准值小的元素时，就把它交换到 `i+1` 的位置。这样遍历结束后，所有比基准小的元素都会被移到前面，最后再把基准值放到 `i+1` 的位置，并返回这个位置作为新的划分点。这个位置将作为接下来递归排序的分界线。

(ii). quicksort 函数

```
void quickSort(std::vector<int>& arr, int low, int high, int depth = 0) {
    printLogicalThreadIdOnce(); // 每个线程仅输出一行编号

    if (low<high) {
        int pi = partition(arr, low, high); // 获取分区位置

        if (depth < MAX_THREAD_DEPTH) {
            // 并行处理左半部分
            std::thread t1([&arr, low, pi, depth]() {
                quickSort(arr, low, pi-1, depth+1);
            });

            // 当前线程处理右半部分
            quickSort(arr, pi+1, high, depth+1);
            t1.join(); // 等待子线程结束
        }
        else {
            // 超过最大深度，转为串行递归
            quickSort(arr, low, pi-1, depth+1);
            quickSort(arr, pi+1, high, depth+1);
        }
    }
}
```

执行快速排序的递归逻辑，并在适当的深度启用多线程以实现并行加速。函数首先调用 `printLogicalThreadIdOnce()` 来记录当前线程的执行信息。通过 `partition()` 获取当前数组区间的分界点 `pi`。若当前递归深度 `depth` 小于允许的最大深度 `MAX_THREAD_DEPTH`，说明线程总数还未达到设定上限，因此程序会为左半部分新建一个线程执行递归排序，而右半部分则继续由当前线程处理。子线程完成后通过 `join()` 等待其结束。如果当前深度已达到上限，程序则退回为串行递归，分别处理左右子区间。

4. 实验结果

(1). C++ 实现双调归并排序（1_bitonic_sort.cpp）

数据规模为 1048576 (2^{20})，线程数分别为 1、2、4、8：

```
chenih@Chenih5: ~/Parallel Algorithms/Lab1
chenih@Chenih5:~/Parallel Algorithms/Lab1$ g++ -std=c++17 1_bitonic_sort.cpp -o 1_bitonic_sort -pthread
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 1048576 1
Thread 0 executed.
Execution time: 1.07591s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 1048576 2
Thread 0 executed.
Thread 1 executed.
Execution time: 0.78834s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 1048576 4
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Execution time: 0.759858s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 1048576 8
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Thread 4 executed.
Thread 5 executed.
Thread 6 executed.
Thread 7 executed.
Execution time: 0.931174s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$
```

数据规模为 2097152 (2^{21})，线程数分别为 1、2、4、8：

```
chenih@Chenih5: ~/Parallel Algorithms/Lab1
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 2097152 1
Thread 0 executed.
Execution time: 2.22942s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 2097152 2
Thread 0 executed.
Thread 1 executed.
Execution time: 1.47934s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 2097152 4
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Execution time: 1.97506s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 2097152 8
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Thread 4 executed.
Thread 5 executed.
Thread 6 executed.
Thread 7 executed.
Execution time: 1.93254s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$
```

数据规模为 4194304 (2^{22})，线程数分别为 1、2、4、8：

```
chenih@Chenih5: ~/Parallel Algorithms/Lab1
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 4194304 1
Thread 0 executed.
Execution time: 4.96546s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 4194304 2
Thread 0 executed.
Thread 1 executed.
Execution time: 3.24917s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 4194304 4
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Execution time: 3.58031s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 4194304 8
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Thread 4 executed.
Thread 5 executed.
Thread 6 executed.
Thread 7 executed.
Execution time: 3.68897s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$
```

数据规模为 8388608 (2^{23})，线程数分别为 1、2、4、8：

```
chenih@Chenih5: ~/Parallel Algorithms/Lab1
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 8388608 1
Thread 0 executed.
Execution time: 11.6868s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 8388608 2
Thread 0 executed.
Thread 1 executed.
Execution time: 6.34765s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 8388608 4
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Execution time: 8.44626s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 8388608 8
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Thread 4 executed.
Thread 5 executed.
Thread 6 executed.
Thread 7 executed.
Execution time: 7.7626s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$
```

数据规模为 16777216 (2^{24})，线程数分别为 1、2、4、8：

```
chenih@Chenih5: ~/Parallel Algorithms/Lab1
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 16777216 1
Thread 0 executed.
Execution time: 22.4515s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 16777216 2
Thread 0 executed.
Thread 1 executed.
Execution time: 18.7521s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 16777216 4
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Execution time: 22.0851s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./1_bitonic_sort 16777216 8
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Thread 4 executed.
Thread 5 executed.
Thread 6 executed.
Thread 7 executed.
Execution time: 22.079s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$
```

(2). C++ 实现并行快速排序 (2_parallel_quicksort.cpp)

数据规模为 1048576 (2^{20})，线程数分别为 1、2、4、8：

```
chenih@Chenih5: ~/Parallel Algorithms/Lab1
chenih@Chenih5:~/Parallel Algorithms/Lab1$ g++ -std=c++17 2_parallel_quicksort.cpp -o 2_parallel_quicksort -pthread
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 1048576 1
Thread 0 executed.
Execution time: 0.548604s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 1048576 2
Thread 0 executed.
Thread 1 executed.
Execution time: 0.683094s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 1048576 4
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Execution time: 0.614866s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 1048576 8
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Thread 4 executed.
Thread 5 executed.
Thread 6 executed.
Thread 7 executed.
Execution time: 0.668085s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$
```

数据规模为 2097152 (2^{21})，线程数分别为 1、2、4、8：

```
chenih@Chenih5: ~/Parallel Algorithms/Lab1
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 2097152 1
Thread 0 executed.
Execution time: 0.640146s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 2097152 2
Thread 0 executed.
Thread 1 executed.
Execution time: 0.620169s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 2097152 4
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Execution time: 0.654283s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 2097152 8
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Thread 4 executed.
Thread 5 executed.
Thread 6 executed.
Execution time: 0.686331s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$
```

数据规模为 4194304 (2^{22})，线程数分别为 1、2、4、8：

```
chenih@Chenih5: ~/Parallel Algorithms/Lab1
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 4194304 1
Thread 0 executed.
Execution time: 1.3528s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 4194304 2
Thread 0 executed.
Thread 1 executed.
Execution time: 1.47189s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 4194304 4
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Execution time: 1.37407s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 4194304 8
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Thread 4 executed.
Thread 5 executed.
Thread 6 executed.
Execution time: 1.46599s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$
```

数据规模为 8388608 (2^{23})，线程数分别为 1、2、4、8：

```
chenih@Chenih5: ~/Parallel Algorithms/Lab1
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 8388608 1
Thread 0 executed.
Execution time: 2.60331s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 8388608 2
Thread 0 executed.
Thread 1 executed.
Execution time: 2.7715s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 8388608 4
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Execution time: 2.89463s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 8388608 8
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Thread 4 executed.
Thread 5 executed.
Thread 6 executed.
Thread 7 executed.
Execution time: 3.22872s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$
```

数据规模为 16777216 (2^{24})，线程数分别为 1、2、4、8：

```
chenih@Chenih5: ~/Parallel Algorithms/Lab1
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 16777216 1
Thread 0 executed.
Execution time: 6.30964s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 16777216 2
Thread 0 executed.
Thread 1 executed.
Execution time: 7.18762s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 16777216 4
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Execution time: 8.00679s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$ ./2_parallel_quicksort 16777216 8
Thread 0 executed.
Thread 1 executed.
Thread 2 executed.
Thread 3 executed.
Thread 4 executed.
Thread 5 executed.
Thread 6 executed.
Execution time: 6.74848s
Sorted correctly.
chenih@Chenih5:~/Parallel Algorithms/Lab1$
```


(3). 运行时间

在 `1_bitonic_sort.cpp` 的测试中，对于 2^{20} 数据规模，单线程的执行时间为 1.07591 秒，使用 2 线程的执行时间降至 0.78834 秒，明显表现出并行化带来的加速效果。当线程数增加至 8 时，执行时间反而略微回升至 0.931174 秒，说明在初期，增加线程数能够显著加速排序过程，但随着线程数的增多，性能提升开始逐渐减缓。这种现象可能是由于线程调度和管理开销的增加，导致了线程的调度时间和同步开销超过了计算任务本身的执行时间。

对于其他数据规模，运行结果呈现出类似的趋势。随着线程数的增加，执行时间显著下降，但当线程数过多时，加速效果逐渐饱和，性能提升的空间变得有限。所以在并行化的过程中，线程数并非越多越好，过多的线程可能反而因为增加的管理开销和资源竞争，导致效率的降低。

在 `2_parallel_quicksort.cpp` 的测试中， 2^{20} 数据规模下的单线程执行时间为 0.54860 秒，而在 2 线程时，执行时间为 0.68309 秒，虽然有加速效果，但相对较小。这表明，较小的数据规模下，并行化并未带来显著的性能提升，可能是由于线程调度开销和线程管理造成的额外负担。当线程数进一步增加到 8 时，执行时间略微回升至 0.66808 秒，在较小规模数据上，线程数的增加未能有效带来期望中的性能提升。小规模数据的排序任务本身所需的计算量较少，导致线程调度和管理开销相对较大，进而抵消了多线程的优势。

(4). 加速比计算

加速比计算公式： $Speedup = \frac{T_{single}}{T_{parallel}}$ 。

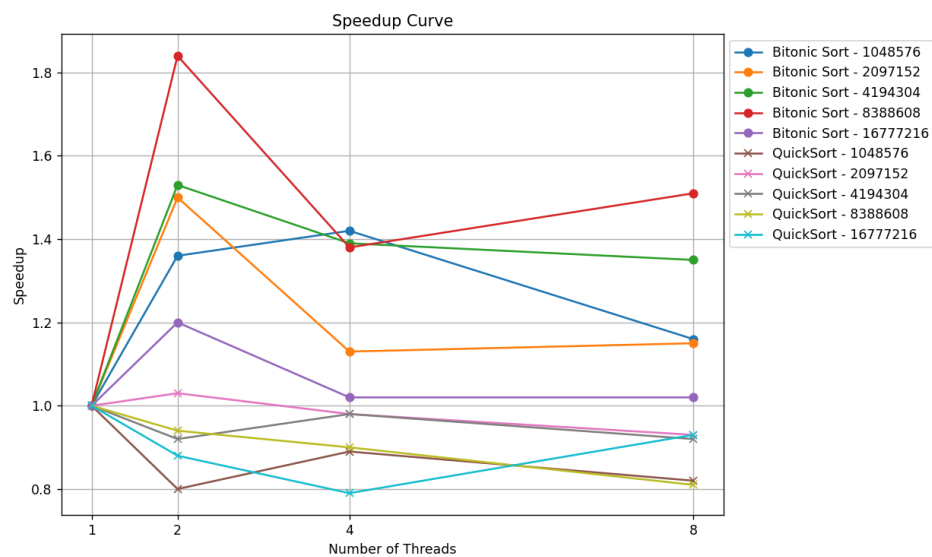
(i). `1_bitonic_sort.cpp`:

线程数\数据规模	2^{20}	2^{21}	2^{22}	2^{23}	2^{24}
1	1.00	1.00	1.00	1.00	1.00
2	1.36	1.50	1.53	1.84	1.20
4	1.42	1.13	1.39	1.38	1.02
8	1.16	1.15	1.35	1.51	1.02

(ii). `2_parallel_quicksort.cpp`:

线程数\数据规模	2^{20}	2^{21}	2^{22}	2^{23}	2^{24}
1	1.00	1.00	1.00	1.00	1.00
2	0.80	1.03	0.92	0.94	0.88
4	0.89	0.98	0.98	0.90	0.79
8	0.82	0.93	0.92	0.81	0.93

加速比曲线：



从图中可以看出，bitonic sort 在数据规模较小时（如 2^{20} ）表现出明显的加速效果。quickSort 在较小数据规模时，线程数的增加对加速比影响较小，特别是从 2 线程到 8 线程时，加速比变化较小，这可能是因为 quickSort 本身是一个递归深度较浅的算法，线程数增加并不会显著提高性能。

5. 分析与总结

(1). 负载均衡分析

根据两个程序的输出结果，可以确认所有线程都得到了合理调度。每次执行时，输出的线程编号（如 Thread 0 到 Thread 7）表示了各线程的参与情况。通过这些输出，可以看出每个线程都在并行计算中执行了任务，确保了负载均衡。没有线程处于空闲状态，这表明任务被均匀地分配到每个线程上，有效避免了线程饥饿或空闲的现象，提升了并行化的效率。

(2). 算法对比分析

双调归并排序在并行计算中具有显著优势，尤其在大规模数据集时，能够通过并行化递归过程有效提高性能。双调归并排序可以有效地分割任务并利用多个处理单元来加速合并操作。由于合并步骤可以并行处理，当数据规模足够大时，性能提升明显。算法稳定性强，适合处理复杂的数据集。它的缺点是内存开销较大，尤其在内存资源有限时可能成为瓶颈。而且在数据规模较小或线程过多时，线程调度和同步的开销可能会抵消并行化带来的加速效果，导致性能反而下降。

并行快速排序的平均时间复杂度为 $O(n \log n)$ ，它通过分治法将数据分为多个子部分进行排序，能够有效地减少排序所需的时间。通过并行化快速排序，可以将多个递归任务分配给不同的线程。并行快速排序的内存效率相对较高。然而快速排序在某些数据集上表现不稳定，例如在数据已经接近排序时，它可能会退化为较差的时间复杂度，导致性能下降。此外，在较小数据集上，过多的线程可能会导致线程调度开销超出计算任务的实际收益，减少了并行化的效果。

(3). 适应性

双调归并排序和并行快速排序在多核 CPU 和 GPU 上表现有所不同。双调归并排序在 GPU 上特别适合，因为它具有高度并行性，能够充分利用 GPU 的大量处理核心进行数据分割和合并。对于大规模数据集，GPU 能够加速排序的每一个递归阶段，而 CPU 多核环境也能有效并行化任务。然而，在较小数据规模或内存资源有限的情况下，双调归并排序的优势可能不如并行快速排序明显。并行快速排序在大部分现代多核 CPU 环境中也能够获得良好的加速效果，尤其是在数据规模较大时。对于 GPU 而言，虽然并行快速排序也可以并行化，但它通常不像双调归并排序那样能显著地提高性能，因为快速排序在并行化时需要处理较为复杂的同步问题。

(4). 性能分析与影响因素

并行算法的性能受线程数、数据规模、调度机制和硬件资源共同影响。合理配置线程数能显著提升速度，但存在瓶颈，初期增加线程可充分利用多核资源，但超过硬件承受能力后，线程竞争和调度开销反而降低效率，例如线程数从 2 增至 8 时，执行时间可能不降反升。数据规模也对性能十分重要，小数据时线程管理成本可能超过计算收益，而大数据时并行优势更明显。此外，递归深度控制需权衡，过浅无法发挥多核性能，过深则线程过多导致资源争抢。因此，优化需综合考虑线程配置、任务划分粒度、内存访问效率等因素，才能在不同硬件环境下实现最佳性能。

(5). 改进方案

在本次实验中，尽管并行算法在一定程度上提升了排序性能，但仍存在进一步优化的空间。当前实现中，线程的创建基于递归深度进行控制，线程调度是静态的。可以考虑采用任务池或任务队列的方式，将可执行任务动态分配给空闲线程，从而提高线程利用率，减少空闲时间。或者可以考虑：动态划分任务，根据数组段大小调整分配线程，也可以使用工作窃取（work stealing）策略，让空闲线程主动寻找其他任务执行，以保持系统负载均衡。

当前采用的是根据线程数自动设置 MAX_THREAD_DEPTH，可以考虑根据数据规模和系统线程数进行更灵活的计算；在数据规模较小时，可以设定一个阈值，当子数组大小低于该阈值时，改用串行排序算法，以减少线程创建开销；排序过程中频繁访问内存，如果能改进数据访问的局部性，例如避免跨线程访问同一缓存行，也有助于性能提升。

6. 实验感想

通过这次实验，我对多线程编程在算法中的应用有了更直观的认识。比如在递归实现的排序算法中，如何合理拆分任务、控制线程数量，同时避免负载不均衡的问题。以双调归并排序和并行快速排序为例，同样的数据规模下，两者的加速效果差异明显，这让我意识到，并行化不仅是“多开线程”，更需要结合算法特点、线程调度策略和数据划分方式共同优化。使用 thread 和互斥锁时，

我对多线程的同步机制有了更具体的理解，比如如何避免数据竞争、怎么用线程编号跟踪执行状态。最后通过对比不同线程数和数据量的运行时间，我了解到性能瓶颈和加速比的规律。比如小数据量时线程创建的开销可能超过计算收益，而大数据量时多核并行才能真正提速。这让我对并行算法有了更深的理解。

7. 参考资料

1. https://blog.csdn.net/ddppqq/article/details/15029751?ops_request_misc=%257B%2522request%255Fid%2522%253A%2522a37d4d999d795089a84d65f1eec5f872%2522%252C%2522scm%2522%253A%25220140713.130102334..%2522%257D&request_id=a37d4d999d795089a84d65f1eec5f872&biz_id=0&utm_medium=distribute.pc_search_result.none-task-blog-2~all~baidu_landing_v2~default-6-15029751-null-null.142^v102^pc_search_result_base5&utm_term=%E5%8F%8C%E8%B0%83%E5%BD%92%E5%B9%B6%E6%8E%92%E5%BA%8F&spm=1018.2226.3001.4187
2. https://blog.csdn.net/qq_47814951/article/details/138341713?ops_request_misc=%257B%2522request%255Fid%2522%253A%2522a2e1e8916eb0175f2480e6acdc0bda90%2522%252C%2522scm%2522%253A%25220140713.130102334..%2522%257D&request_id=a2e1e8916eb0175f2480e6acdc0bda90&biz_id=0&utm_medium=distribute.pc_search_result.none-task-blog-2~all~sobaiduend~default-2-138341713-null-null.142^v102^pc_search_result_base5&utm_term=%E5%B9%B6%E8%A1%8C%E5%BF%AB%E9%80%9F%E6%8E%92%E5%BA%8F&spm=1018.2226.3001.4187